

An Analysis of Sentence embedding

Sentence embedding

Sentence embedding -- Part 1

Applications In recent years, sentence embedding has seen a growing level of interest due to its applications in natural language queryable knowledge bases through the usage of vector indexing for semantic search. LangChain for instance utilizes sentence transformers for purposes of indexing documents. In particular, an indexing is generated by generating embeddings for chunks of documents and storing (document chunk, embedding) tuples. Then given a query in natural language, the embedding for the query can be generated. A top k similarity search algorithm is then used between the query embedding and the document chunk embeddings to retrieve the most relevant document chunks as context information for question answering tasks. This approach is also known formally as retrieval-augmented generation. Though not as predominant as BERTScore, sentence embeddings are commonly used for sentence similarity evaluation which sees common use for the task of optimizing a Large language model's generation parameters is often performed via comparing candidate sentences against reference sentences. By using the cosine-similarity of the sentence embeddings of candidate and reference sentences as the evaluation function, a grid-search algorithm can be utilized to automate hyperparameter optimization.

Sentence embedding -- Part 2

Evaluation A way of testing sentence encodings is to apply them on Sentences Involving Compositional Knowledge (SICK) corpus for both entailment (SICK-E) and relatedness (SICK-R). In the best results are obtained using a BiLSTM network trained on the Stanford Natural Language Inference (SNLI) Corpus. The Pearson correlation coefficient for SICK-R is 0.885 and the result for SICK-E is 86.3. A slight improvement over previous scores is presented in: SICK-R: 0.888 and SICK-E: 87.8 using a concatenation of bidirectional Gated recurrent unit.

Sentence embedding -- Part 3

In natural language processing, a sentence embedding is a representation of a sentence as a vector of numbers which encodes meaningful semantic information. State of the art embeddings are based on the learned hidden layer representation of dedicated sentence transformer models. BERT pioneered an approach involving the use of a dedicated [CLS] token prepended to the beginning of each sentence inputted into the model; the final hidden state vector of this token encodes information about the sentence and can be fine-tuned for use in sentence classification tasks. In practice however, BERT's sentence embedding with the [CLS] token achieves poor performance, often worse than simply averaging non-contextual word

embeddings. SBERT later achieved superior sentence embedding performance by fine tuning BERT's [CLS] token embeddings through the usage of a siamese neural network architecture on the SNLI dataset. Other approaches are loosely based on the idea of distributional semantics applied to sentences. Skip-Thought trains an encoder-decoder structure for the task of neighboring sentences predictions; this has been shown to achieve worse performance than approaches such as InferSent or SBERT. An alternative direction is to aggregate word embeddings, such as those returned by Word2vec, into sentence embeddings. The most straightforward approach is to simply compute the average of word vectors, known as continuous bag-of-words (CBOW). However, more elaborate solutions based on word vector quantization have also been proposed. One such approach is the vector of locally aggregated word embeddings (VLAW), which demonstrated performance improvements in downstream text classification tasks.